

Table 1: Comparison of row-based and columnar databases

Characteristics	Row-based	Columnar
Storage pattern	Row by row	Column by column
Read pattern	Fetch full rows	Fetch specific columns
Write speed	Very fast	Moderate
Aggregation speed	Slow (reads all columns)	Very fast (reads only needed cols)
Compression	Low ratio	High ratio (same-type data)
Best workload	Transactional (OLTP)	Analytical (OLAP)
Examples	PostgreSQL, MySQL, Oracle	Redshift, BigQuery, ClickHouse

When to use each model

Use row-based databases when:

- Your workload is dominated by inserts, updates, and deletes (OLTP).
- Queries frequently retrieve whole rows by primary key or a small set of rows.
- You need strong ACID guarantees with low-latency writes.
- Your application is a web backend, CRM, ERP, or payment system.

Use columnar databases when:

- You run analytical queries (OLAP) that aggregate millions or billions of rows.
- Queries touch only a few columns out of many (e.g., *SELECT region, SUM(revenue) ...*).
- Storage cost and I/O bandwidth are primary concerns.
- You need a data warehouse, BI platform, or real-time analytics layer.

Row-based example: PostgreSQL

PostgreSQL is one of the most popular row-based databases. Below is a schema and query pattern suited to transactional OLTP workloads:

```
-- Create a row-based orders table
CREATE TABLE orders (
  order_id SERIAL PRIMARY KEY,
  customer_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price NUMERIC(10,2),
  created_at TIMESTAMPTZ DEFAULT now()
);
```

```
-- Index for fast point lookups (row-based strength)
CREATE INDEX idx_orders_customer ON orders (customer_id);
```

```
-- Efficient point lookup – reads a single row
SELECT * FROM orders WHERE order_id = 42;
```

```
-- Insert is fast because one contiguous row is written
INSERT INTO orders (customer_id, product_id, quantity, unit_price)
VALUES (101, 55, 2, 29.99);
```

PostgreSQL also provides the *pg_column_store* extension (and Citus) for hybrid scenarios, but its native heap storage is fully row-oriented.

Columnar example: ClickHouse

ClickHouse is an open source columnar OLAP database that can ingest billions of rows and return aggregations in milliseconds.

```
-- ClickHouse columnar table
CREATE TABLE orders (
  order_id UInt64,
  customer_id UInt32,
  product_id UInt32,
  quantity UInt16,
  unit_price Decimal(10,2),
  created_at DateTime
)
ENGINE = MergeTree()
ORDER BY (created_at, customer_id);

-- Analytical aggregation – only reads unit_price & quantity
SELECT
  toStartOfMonth(created_at) AS month,
  sum(quantity * unit_price) AS total_revenue
FROM orders
WHERE created_at >= '2025-01-01'
GROUP BY month
ORDER BY month;
```

ClickHouse reads only the two columns involved, applies vectorised SIMD operations across millions of rows, and returns results orders of magnitude faster than the equivalent query in PostgreSQL on the same hardware.

Performance characteristics at scale

Consider a 1-billion-row sales table with 20 columns. A query computing monthly revenue involves only two columns. Table 2 compares the performance of PostgreSQL and Clickhouse based on various metrics.